

VALIDACIÓN CRUZADA Y OPTIMIZACIÓN DE PARÁMETROS

Adriana Haydeé Contreras Peruyero

*Escuela Nacional de Estudios Superiores Unidad Juriquilla,
UNAM*

haydeeperuyero@unam.mx

Temas Selectos de Aprendizaje
de Máquina
Semestre 2027-1



ENES
JURIQUILLA

1 Parámetros e Hiperparámetros

2 Entrenamiento, Validación y prueba

3 Cross-Validation

4 Grid Search

¿Dónde nos quedamos?

En la clase anterior vimos que modelos como Gradient Boosting y XGBoost dependen de varios **hiperparámetros**:

`learning_rate, n_estimators, max_depth, subsample,...`

Distintas elecciones pueden producir modelos muy diferentes.

Dos preguntas

1. ¿Cómo estimamos qué tan bien generaliza una configuración?
2. ¿Cómo buscamos una buena combinación de hiperparámetros?

PARÁMETROS E HIPERPARÁMETROS

Parámetros e hiperparámetros

Parámetros

Se **aprenden a partir de los datos** durante el entrenamiento.

Por ejemplo, en regresión lineal:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

Los coeficientes

$$\beta_0, \beta_1, \dots, \beta_p$$

son parámetros.

Hiperparámetros

Controlan la estructura del modelo o el proceso de aprendizaje.

Se especifican **antes de entrenar**.

Ejemplos:

`k`, `max_depth`, `learning_rate`.

Ejemplos de hiperparámetros

Modelo	Hiperparámetros
k -NN	número de vecinos k
Árbol de decisión	<code>max_depth</code> , <code>min_samples_leaf</code>
Random Forest	<code>n_estimators</code> , <code>max_features</code>
Gradient Boosting	<code>learning_rate</code> , <code>n_estimators</code>
XGBoost	<code>max_depth</code> , <code>subsample</code> , <code>reg_lambda</code>

Importante

Los parámetros los aprende el algoritmo.

Los hiperparámetros forman parte de las decisiones que debemos tomar al construir el modelo.

El problema de selección

Representemos un algoritmo con hiperparámetros λ como

$$A_{\lambda}.$$

Al entrenarlo sobre los datos obtenemos

$$\hat{f}_{\lambda} = A_{\lambda}(\mathcal{D}_{\text{train}}).$$

Queremos encontrar una configuración

$$\lambda = (\lambda_1, \lambda_2, \dots, \lambda_p)$$

que produzca un modelo con buena capacidad de **generalización**.

Pero...

¿Cómo sabemos si una configuración generaliza bien?

Entrenar no es lo mismo que evaluar

Supongamos que entrenamos $\hat{f}$ utilizando

$$\mathcal{D}_{\text{train}}.$$

Podemos calcular el error sobre esas mismas observaciones:

$$\hat{R}_{\text{train}} = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{f}(\mathbf{x}_i)).$$

Un error de entrenamiento pequeño significa que el modelo reproduce bien los datos utilizados para ajustarlo.

Entrenar no es lo mismo que evaluar

Supongamos que entrenamos $\hat{f}$ utilizando

$$\mathcal{D}_{\text{train}}.$$

Podemos calcular el error sobre esas mismas observaciones:

$$\hat{R}_{\text{train}} = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{f}(\mathbf{x}_i)).$$

Un error de entrenamiento pequeño significa que el modelo reproduce bien los datos utilizados para ajustarlo.

¿Pero eso es realmente lo que nos interesa?

Nuestro verdadero interés es el desempeño sobre **observaciones nuevas**.

Idealmente quisiéramos conocer

$$R(f) = \mathbb{E}_{(X,Y)} [L(Y, f(X))].$$

Este es el **error de generalización**.

El problema es que no conocemos la distribución que genera los datos.

Consecuencia

Necesitamos estimar el error de generalización utilizando observaciones que no hayan sido utilizadas para ajustar el modelo.

El peligro del error de entrenamiento

Un modelo suficientemente flexible puede obtener

$$\hat{R}_{\text{train}} \approx 0$$

y, sin embargo, producir malas predicciones sobre observaciones nuevas.

buen ajuste a train $\nRightarrow$ buena generalización

Importante

No debemos elegir hiperparámetros simplemente buscando la configuración con menor error de entrenamiento.

ENTRENAMIENTO, VALIDACIÓN Y PRUEBA

Una primera estrategia consiste en dividir

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{validation}} \cup \mathcal{D}_{\text{test}}.$$

Conjunto	Función	Pregunta
Train	Aprender	¿Qué parámetros?
Validation	Decidir	¿Qué configuración?
Test	Evaluar al final	¿Cómo generaliza?

Training set: aprender

El conjunto de entrenamiento se utiliza para ajustar los **parámetros** del modelo.

$$\mathcal{D}_{\text{train}} \longrightarrow \hat{f}_{\lambda}.$$

Por ejemplo:

- ▶ coeficientes de una regresión;
- ▶ divisiones de un árbol;
- ▶ valores de sus hojas;
- ▶ árboles que forman un ensamble.

La configuración λ se considera fija durante este entrenamiento.

Validation set: decidir

Supongamos que queremos comparar

$$\lambda_1, \lambda_2, \dots, \lambda_M.$$

Entrenamos los modelos correspondientes utilizando $\mathcal{D}_{\text{train}}$ y los evaluamos sobre $\mathcal{D}_{\text{validation}}$.

Si utilizamos una medida de error:

$$\lambda^* = \arg \min_{\lambda} \hat{R}_{\text{validation}}(\hat{f}_{\lambda})$$

Función del validation set

Nos permite **tomar decisiones** sobre el procedimiento de modelado.

Test set: evaluar al final

El conjunto de prueba se reserva para estimar el desempeño final del procedimiento seleccionado.

Train $\longrightarrow$ aprender

Validation $\longrightarrow$ decidir

Test $\longrightarrow$ evaluar al final

Regla fundamental

El test set debe permanecer fuera del proceso de selección.

¿Por qué no seleccionar con el test set?

Supongamos que hacemos:

$$\lambda_1 \longrightarrow \text{test}$$

No nos gusta el resultado, entonces cambiamos:

$$\lambda_2 \longrightarrow \text{test}$$

y continuamos hasta obtener un resultado satisfactorio:

$$\lambda_3 \longrightarrow \text{test} \longrightarrow \dots$$

¿Por qué no seleccionar con el test set?

Supongamos que hacemos:

$$\lambda_1 \longrightarrow \text{test}$$

No nos gusta el resultado, entonces cambiamos:

$$\lambda_2 \longrightarrow \text{test}$$

y continuamos hasta obtener un resultado satisfactorio:

$$\lambda_3 \longrightarrow \text{test} \longrightarrow \dots$$

¿Qué ocurrió?

Aunque no entrenamos directamente con el test, **nuestras decisiones dependieron de él.**

El test deja de representar datos nuevos

Si consultamos repetidamente el test para modificar:

- ▶ hiperparámetros;
- ▶ selección de variables;
- ▶ preprocesamiento;
- ▶ arquitectura del modelo;
- ▶ cualquier otra decisión;

estamos incorporando indirectamente información del test.

usar test para decidir $\Rightarrow$ test deja de ser evaluación final
--

¿Es suficiente una sola partición?

Supongamos que dividimos:

70% train, 15% validation, 15% test.

El desempeño sobre validation depende de **qué observaciones terminaron por azar en ese 15%**.

Dos particiones distintas pueden producir:

resultados distintos

e incluso cambiar cuál configuración parece preferible.

El tamaño del conjunto de validación

Existe un compromiso:

Validation pequeño

Disponemos de más observaciones para entrenar, pero la estimación del desempeño puede ser **inestable**.

Validation grande

La evaluación puede ser más informativa, pero disponemos de menos observaciones para **entrenar**.

¿Podemos utilizar mejor los datos disponibles?

CROSS-VALIDATION

k -fold Cross-Validation

Dividimos los datos destinados al desarrollo del modelo en

$$k$$

subconjuntos aproximadamente del mismo tamaño:

$$\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_k.$$

Estos subconjuntos se llaman **folds**.

En cada iteración:

- ▶ un fold funciona como validación;
- ▶ los $k - 1$ folds restantes se utilizan para entrenar.

Cada fold se utiliza **exactamente una vez** como validación.

Ejemplo: 5-fold Cross-Validation

	$\mathcal{D}_1$	$\mathcal{D}_2$	$\mathcal{D}_3$	$\mathcal{D}_4$	$\mathcal{D}_5$
1	V	T	T	T	T
2	T	V	T	T	T
3	T	T	V	T	T
4	T	T	T	V	T
5	T	T	T	T	V

T = training, V = validation.

Cada observación participa en validación una vez y en entrenamiento $k - 1$ veces.

Cross-Validation produce varios resultados

Después de k iteraciones obtenemos k valores de desempeño:

$$s_1, s_2, \dots, s_k.$$

Podemos resumir el desempeño promedio mediante

$$\bar{s}_{CV} = \frac{1}{k} \sum_{j=1}^k s_j$$

y su variabilidad mediante

$$s_{CV} = \sqrt{\frac{1}{k-1} \sum_{j=1}^k (s_j - \bar{s}_{CV})^2}$$

No queremos solamente un promedio

Supongamos que dos configuraciones producen:

Modelo A: 0.81, 0.80, 0.82, 0.79, 0.81

Modelo B: 0.70, 0.91, 0.72, 0.90, 0.80

Ambas podrían tener un promedio parecido.

Pero su comportamiento entre particiones es muy diferente.

Idea clave

Cross-Validation permite observar:

desempeño promedio

+

variabilidad

¿Cuántos folds debemos utilizar?

Dos elecciones comunes son

$$k = 5 \quad \text{y} \quad k = 10.$$

En cada iteración utilizamos para entrenar una proporción

$$\frac{k-1}{k}.$$

Por tanto:

$$k = 5 \quad \Rightarrow \quad 80\% \text{ para entrenamiento,}$$

$$k = 10 \quad \Rightarrow \quad 90\% \text{ para entrenamiento.}$$

Más folds también cuestan más

Si entrenar una configuración requiere aproximadamente un tiempo T ,

entonces k -fold Cross-Validation requiere aproximadamente

$$\boxed{kT}.$$

Por tanto, aumentar k implica utilizar más datos para entrenar en cada iteración, pero también aumentar el costo computacional.

Regla práctica

No existe un valor universalmente óptimo.

En muchas aplicaciones,

$$k = 5 \quad \text{o} \quad k = 10$$

ofrecen un compromiso razonable.

¿Podemos dividir siempre aleatoriamente?

No.

La estrategia de validación debe respetar la estructura del problema.

Debemos preguntarnos:

- ▶ ¿las clases están desbalanceadas?
- ▶ ¿existen observaciones relacionadas?
- ▶ ¿hay múltiples mediciones del mismo individuo?
- ▶ ¿los datos tienen un orden temporal?

Idea clave

Cross-Validation no significa simplemente dividir aleatoriamente los datos.

Stratified k -fold

En clasificación puede ser importante conservar aproximadamente la **proporción de las clases** en cada fold.

Por ejemplo, si tenemos:

80% clase 0, 20% clase 1,

buscamos que cada fold tenga aproximadamente:

80%/20%.

¿Por qué?

Una partición completamente aleatoria podría producir folds con muy pocas observaciones de la clase minoritaria.

A veces las observaciones **no son independientes**.

Podemos tener varias mediciones provenientes de:

- ▶ un mismo paciente;
- ▶ un mismo individuo;
- ▶ una misma muestra biológica;
- ▶ un mismo sitio;
- ▶ una misma cepa;
- ▶ una misma unidad experimental.

Observaciones relacionadas

Supongamos que tenemos varias mediciones de un mismo individuo.

Si algunas aparecen en entrenamiento y otras en validación:

$$\text{individuo A} \longrightarrow \begin{cases} \text{train} \\ \text{validation} \end{cases}$$

el modelo puede disponer indirectamente de información muy relacionada con las observaciones que intenta predecir.

Importante

El desempeño estimado puede ser **artificialmente alto**. Este es un ejemplo de **data leakage**.

Group k -fold mantiene juntas las observaciones pertenecientes al mismo grupo.

Validación con datos temporales

Si los datos tienen orden temporal, una partición aleatoria puede ser inadecuada.

No queremos utilizar

futuro

para predecir

pasado.

Una estrategia posible es:

$$\underbrace{\mathcal{D}_1}_{\text{train}} \longrightarrow \underbrace{\mathcal{D}_2}_{\text{validation}},$$

después

$$\underbrace{\mathcal{D}_1 \cup \mathcal{D}_2}_{\text{train}} \longrightarrow \underbrace{\mathcal{D}_3}_{\text{validation}},$$

y continuar de manera semejante.

¿Qué estrategia de validación utilizar?

No existe una estrategia adecuada para todos los problemas.

Observación

La estrategia de validación debe imitar, en la medida de lo posible, la forma en que el modelo será utilizado con datos nuevos.

Por ello, antes de elegir los folds debemos comprender:

- ▶ cómo se generaron los datos;
- ▶ qué observaciones están relacionadas;
- ▶ qué tipo de predicción queremos realizar.

De la validación a la optimización

Supongamos que nuestro modelo depende de

$$\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \dots, \lambda_p).$$

Definimos un espacio de búsqueda:

$$\Lambda = \Lambda_1 \times \Lambda_2 \times \dots \times \Lambda_p.$$

Para cada configuración

$$\boldsymbol{\lambda} \in \Lambda$$

podemos estimar su desempeño mediante Cross-Validation.

Optimización de hiperparámetros

Si nuestra métrica es un **error**, buscamos:

$$\lambda^* = \arg \min_{\lambda \in \Lambda} CV(\lambda)$$

Si utilizamos una métrica donde valores mayores son mejores:

$$\lambda^* = \arg \max_{\lambda \in \Lambda} CV(\lambda)$$

Optimización de hiperparámetros

Es el proceso de buscar una configuración que produzca un desempeño adecuado de acuerdo con un criterio de evaluación establecido.

GRID SEARCH

Grid Search

Una estrategia directa consiste en especificar varios valores para cada hiperparámetro y evaluar **todas las combinaciones**.

Por ejemplo:

$$n_estimators \in \{100, 200, 500\},$$

$$learning_rate \in \{0.01, 0.1, 0.2\},$$

$$max_depth \in \{2, 3, 4, 5\}.$$

Esto produce una **cuadrícula de configuraciones**.

El tamaño de la cuadrícula

En el ejemplo anterior tenemos:

$$3 \times 3 \times 4 = \boxed{36}$$

configuraciones.

Si utilizamos 5-fold Cross-Validation:

$$36 \times 5 = \boxed{180}$$

ajustes del modelo.

Importante

Cada configuración debe evaluarse sobre los diferentes folds.

El espacio crece rápidamente

Si tenemos p hiperparámetros y el hiperparámetro j tiene m_j valores posibles:

$$N_{\text{config}} = \prod_{j=1}^p m_j$$

Con k -fold Cross-Validation:

$$N_{\text{fits}} = k \prod_{j=1}^p m_j$$

Si tenemos 6 hiperparámetros con 5 valores cada uno:

$$5^6 = 15\,625$$

configuraciones.

El costo puede crecer muy rápido

Con

15 625

configuraciones y 5-fold Cross-Validation necesitamos:

$$5(15\,625) = \boxed{78\,125}$$

ajustes del modelo.

¿Realmente necesitamos explorar todas esas combinaciones?

Limitaciones de Grid Search

Grid Search es exhaustivo **dentro de la cuadrícula especificada**, pero:

1. el número de combinaciones crece rápidamente;
2. los hiperparámetros continuos deben discretizarse;
3. podemos evaluar muchas regiones poco prometedoras;
4. el costo vuelve a multiplicarse por el número de folds.

Además

Una búsqueda exhaustiva sobre una cuadrícula **no significa** buscar todos los valores posibles del hiperparámetro.

En lugar de evaluar sistemáticamente todas las combinaciones, muestreamos configuraciones aleatorias:

$$\lambda^{(1)}, \lambda^{(2)}, \dots, \lambda^{(B)}.$$

El número B representa nuestro **presupuesto de búsqueda**.

Por ejemplo:

$$B = 50$$

significa evaluar 50 configuraciones.

Controlar el presupuesto

Si evaluamos $B = 50$ configuraciones mediante 5-fold Cross-Validation, necesitamos aproximadamente

$$50 \times 5 = 250$$

entrenamientos.

En Random Search podemos cambiar la pregunta:

¿Cuántas combinaciones existen?

por

¿Cuántas configuraciones puedo permitirme evaluar?

¿Por qué puede funcionar Random Search?

Supongamos que tenemos dos hiperparámetros:

$$\lambda_1 \quad \text{y} \quad \lambda_2.$$

Pero el desempeño depende fuertemente de

$$\lambda_1$$

y muy poco de

$$\lambda_2.$$

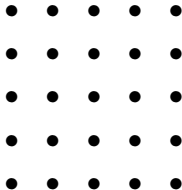
En otras palabras:

λ_1 es importante

λ_2 tiene poco efecto

Grid Search: valores repetidos

Supongamos una cuadrícula 5×5 .



Realizamos 25 evaluaciones.

Sin embargo, solamente exploramos 5 valores diferentes de λ_1 .

Random Search: mayor diversidad

Con las mismas 25 evaluaciones, una búsqueda aleatoria puede explorar muchos más valores distintos de λ_1 .

Si λ_1 es el hiperparámetro que realmente afecta el desempeño, esto puede aprovechar mejor nuestro presupuesto.

Idea clave

Random Search puede explorar más valores distintos de cada hiperparámetro, sin aumentar necesariamente el número total de modelos evaluados.

Grid Search vs. Random Search

Grid Search	Random Search
Evalúa una cuadrícula predefinida.	Muestrea configuraciones aleatoriamente.
Explora todas las combinaciones de la cuadrícula.	Explora un número especificado de configuraciones.
Adecuado para espacios pequeños.	Útil con muchos hiperparámetros.
Costo determinado por la cuadrícula.	Presupuesto controlado directamente.
Puede gastar evaluaciones en dimensiones poco importantes.	Puede explorar más valores distintos de cada hiperparámetro.

Una estrategia práctica

Grid Search y Random Search pueden utilizarse de manera complementaria.

búsqueda amplia $\rightarrow$ región prometedora $\rightarrow$ búsqueda más fina

Por ejemplo, podemos comenzar con Random Search y posteriormente explorar con mayor detalle una región prometedora.

Optimización y costo computacional

En problemas reales existe un compromiso entre

calidad del modelo

y

costo computacional.

La optimización de hiperparámetros no debe entenderse necesariamente como:

probar todo.

Puede entenderse como:

utilizar de manera razonable nuestro presupuesto de búsqueda.

Más allá de Grid Search y Random Search

Existen métodos que utilizan los resultados de evaluaciones anteriores para decidir qué configuraciones explorar después.

Por ejemplo:

- ▶ optimización bayesiana;
- ▶ métodos basados en modelos probabilísticos;
- ▶ Successive Halving;
- ▶ Hyperband;
- ▶ herramientas como Optuna.

Podemos pensar en una evolución conceptual:

manual → Grid → Random → búsqueda adaptativa

¿Cuándo hacemos el preprocesamiento?

Supongamos que queremos estandarizar una característica:

$$z = \frac{x - \mu}{\sigma}.$$

Una posibilidad sería:

1. calcular μ y σ utilizando todos los datos de desarrollo;
2. transformar los datos;
3. después construir los folds.

¿Hay algún problema?

Información que no debería estar disponible

Si calculamos

$$\mu \quad y \quad \sigma$$

antes de construir los folds, las observaciones que posteriormente serán **validation** ya contribuyeron a estimar esos valores.

Validation $\longrightarrow (\mu, \sigma)$ $\longrightarrow$ transformación de Train

La información fluye desde validation hacia el proceso de entrenamiento.

Definición

Existe **data leakage** cuando información que no debería estar disponible durante el entrenamiento participa, directa o indirectamente, en la construcción del modelo.

Puede aparecer al realizar antes de Cross-Validation:

- ▶ escalamiento;
- ▶ imputación de valores faltantes;
- ▶ selección de variables;
- ▶ reducción de dimensionalidad;
- ▶ transformaciones aprendidas de los datos;
- ▶ selección de características utilizando la respuesta.

Preprocesamiento dentro de cada fold

Cualquier transformación que **aprenda información de los datos** debe ajustarse utilizando solamente el training fold.

En cada iteración:

Training fold → ajustar preprocesamiento → entrenar modelo

→ transformar Validation → evaluar

Pipelines

Un **Pipeline** permite considerar como un único procedimiento:

preprocesamiento + modelo

y evaluar todo el procedimiento correctamente dentro de Cross-Validation.

Esto permite integrar:

preprocesamiento+entrenamiento+Cross-Validation+optimización.

Ventaja

Las transformaciones se vuelven a ajustar correctamente dentro de cada fold.

Sugerencia: El flujo correcto

Primero separamos:

$$\mathcal{D} \longrightarrow \mathcal{D}_{\text{train}} + \mathcal{D}_{\text{test}}$$

El conjunto de prueba se reserva.

Dentro de $\mathcal{D}_{\text{train}}$:

k -fold Cross-Validation $\rightarrow$ Grid Search / Random Search

$\rightarrow$ configuración seleccionada

El test permanece cerrado

Durante el desarrollo utilizamos:

Train + Cross-Validation = desarrollo y selección

Solamente cuando las decisiones han terminado utilizamos:

Test = evaluación final

Idea central

El test no participa en la elección de hiperparámetros.

Ideas clave

1. Los **parámetros** se aprenden; los **hiperparámetros** se seleccionan.
2. El error de entrenamiento no mide adecuadamente la **generalización**.
3. El conjunto de **test** se reserva para el final.
4. k -fold Cross-Validation utiliza diferentes particiones para estimar el desempeño.
5. Los folds deben respetar la **estructura de los datos**.
6. Grid Search explora una cuadrícula; Random Search utiliza un **presupuesto de búsqueda**.
7. El preprocesamiento debe ocurrir dentro de cada fold para evitar **data leakage**.